CSCI 6461 | TOPIC 6

Subroutines, Calling Conventions, & the Stack

Function linkage, register preservation rules, stack frame management, and AAPCS64 architectural compliance

Dr. J. Burns
Computer Systems Architecture — AArch64 Reference Platform

Topic 6: High-Level Module Roadmap

1. **Subroutine Linkage & Control Flow**
    - Function calls with `bl`, returns with `ret`, and the Link Register (`x30`).
    - Leaf vs. non-leaf subroutine execution mechanics.
2. **Procedure Call Standard (AAPCS64) & Register Roles**
    - Argument passing, return values, caller-saved vs. callee-saved contracts.
    - Interoperability between compiled C code and hand-written assembly.
3. **The Runtime Stack & Activation Frames**
    - Frame Pointer (`x29`) linkages, stack growth, and 16-byte alignment rules.
    - Constructing function prologues and epilogues.
4. **Assembly Implementation Patterns & Worked Examples**
    - Leaf functions, nested calls, recursive algorithms, and stack-passed parameters.

PART 1

Subroutine Linkage & Control Flow

High-Level Function Execution Model

- **Subroutines** provide modular abstraction, reusability, and clean software architecture.
- To execute a function call, the CPU must satisfy five core requirements:
  1. **Pass Arguments:** Transfer input parameters to the function.
  2. **Transfer Control:** Jump to the function's entry point address.
  3. **Isolate Local State:** Provide memory space for function local variables.
  4. **Return Value:** Pass computed results back to the caller.
  5. **Resume Execution:** Return seamlessly to the instruction following the call.

The Link Register (`x30`) and Branch-with-Link (`bl`)

- Invoking a function requires remembering where to return when it completes.
- **Branch with Link (`bl label`):** Atomically performs two operations:
  1. Saves return address (PC $+ 4$) into the **Link Register (`x30` / `lr`)**.
  2. Branches to target function: PC $\leftarrow$ label.

1. $x_{30} \leftarrow$ **PC** $+ 4$ → **2. PC $\leftarrow$ Target Address**

PC-Relative Calling Range

The `bl` instruction encodes a signed 26-bit immediate offset, allowing direct function calls within a ± 128 MB window of the current instruction.

Subroutine Return Mechanics (`ret`)

- **Return from Subroutine (`ret`):** Redirects execution back to the caller by loading the target address from the Link Register into the Program Counter:

$$\text{PC} \leftarrow x_{30}$$

- Execution resumes at the exact instruction immediately following the original `bl`.

Example: Minimal Leaf Return Sequence

```
caller_func:
  bl helper_func    // x30 = address of next instruction; PC = helper_func
  mov x1, x0        // Execution resumes here after ret!
helper_func:
  add x0, x0, #1    // Compute result
  ret               // PC = x30
```

Leaf vs. Non-Leaf Functions

Leaf Functions

- Calls **no other functions**.
- Link Register (`x30`) is never overwritten.
- Can execute and return directly without saving `x30` to the stack.
- **Zero stack overhead!**

Non-Leaf Functions

- Calls one or more **nested subroutines**.
- Executing a nested `bl` **overwrites** `x30` with the nested return address.
- Must save `x30` to the stack in its prologue before making any nested calls.

The Non-Leaf Rule

If a function calls another function, its own return address in `x30` will be destroyed unless saved to memory upon entry.

Indirect Subroutine Calls (blr)

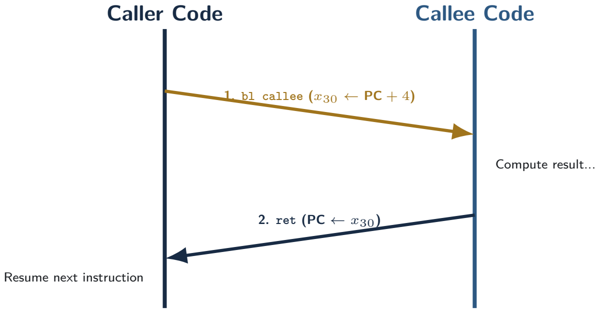
- When the target function address is determined dynamically at runtime, use **Branch with Link to Register (blr xn)**.
- Saves return address ($\text{PC} + 4$) into x30 and jumps to address held in register xn.

Primary Applications

- **C Function Pointers:** `void (*func_ptr)(int) = my_callback;`
- **Virtual Method Tables (vtables):** C++ polymorphism and object dispatch.
- **Shared Library Veneers:** Dynamic linking and Symbol Jump Tables.

```
ldr x2, [x0, #8]    // Load function pointer from vtable
blr x2    // Call function indirectly via register x2
```

Subroutine Call Execution Trace



Control flows smoothly from caller to callee and back using `x30` as the hardware breadcrumb.

PART 2

Procedure Call Standard & Register Roles

Purpose of Procedure Call Standards (AAPCS64)

- **AAPCS64:** Procedure Call Standard for the ARM 64-bit Architecture.
- Defines a strict software contract between caller and callee functions.
- **Why Is an ABI Contract Mandatory?**
  - Allows assembly code to seamlessly call C library functions (`printf`, `malloc`).
  - Enables C compilers (GCC, Clang) to interoperate with assembly routines.
  - Dictates parameter passing, return values, and register preservation rules.

Parameter Passing Registers (x0–x7)

- Up to **8 integer or pointer parameters** are passed directly in registers x0 through x7.

Parameter Index	Register	C Language Example
1st Argument	x0 (w0)	`int64_t a`
2nd Argument	x1 (w1)	`char *str`
3rd Argument	x2 (w2)	`size_t length`
4th–8th Arguments	x3–x7	Additional scalar arguments

Overflow Arguments (>8 Parameters)

Parameters beyond the 8th argument are passed on the runtime stack in reverse order.

Return Value Registers (`x0`–`x7`)

- Functions return scalar results in registers without accessing memory:

Return Data Type	Register Used	Description
32-bit Integer / Boolean	`w0`	Lower 32 bits of register 0
64-bit Integer / Pointer	`x0`	Full 64-bit register 0
128-bit Composite / Struct	`x0` (lower), `x1` (upper)	Split across two registers

Large Struct Returns (> 16 Bytes)

The caller allocates stack memory for the struct and passes an implicit pointer in register **`x8`** (Indirect Result Location).

Caller-Saved Registers (`x0–x17`)

- Also known as **Scratch** or **Volatile** registers.
- The callee function is free to overwrite these registers **without restoring them**.

Register Range	AAPCS64 Role	Callee Behavior
`x0–x7`	Parameter passing / Return values	May overwrite freely
`x8`	Indirect result location pointer	May overwrite freely
`x9–x15`	Temporary caller scratch registers	May overwrite freely
`x16–x17`	Intra-procedure linker scratch (IP0, IP1)	May overwrite freely

Rule for Caller Functions

If a caller needs to preserve values in `x0–x17` across a subroutine call (`bl`), it **must save them to the stack before calling**.

Callee-Saved Registers (x19–x28)

- Also known as **Preserved** or **Non-Volatile** registers.
- Registers x19 through x28 hold long-lived variables across subroutine invocations.

The Callee Contract

If a subroutine modifies any register in x19–x28, it **must save the original value to the stack on entry** and **restore it before returning**.

Callee Entry (Prologue)

```
str x19, [sp, #-16]!
```

Preserves caller's x19 before modifying it.

Callee Exit (Epilogue)

```
ldr x19, [sp], #16
```

Restores original x19 before ret.

Special-Purpose ABI Registers

Register	ABI Name	Architectural Role
`x8`	XR	Indirect result location pointer for struct returns
`x16`, `x17`	IP0, IP1	Intra-procedure call temporaries (used by dynamic linker)
`x18`	PR	Platform register (reserved for OS kernel / thread pointer)
`x29`	FP	Frame Pointer (base anchor for current activation frame)
`x30`	LR	Link Register (holds function return address)
`sp`	SP	Stack Pointer (holds current stack top; 16-byte aligned)

Takeaway: Never use `x18`, `x29`, `x30`, or `sp` as general scratchpad registers!

Register Responsibility Summary

Registers	Role / Usage	Preservation Contract
`x0–x7`	Parameters & Return Values	**Caller-saved** (Callee can overwrite)
`x8–x17`	Scratch / Temporaries	**Caller-saved** (Callee can overwrite)
`x19–x28`	Preserved Local Variables	**Callee-saved** (Callee must restore)
`x29` (`fp`)	Frame Pointer	**Callee-saved** (Callee must restore)
`x30` (`lr`)	Link Register	**Callee-saved** (if non-leaf)
`sp`	Stack Pointer	**Preserved** (Must balance exactly)

PART 3

The Runtime Stack & Activation Frames

The Stack Memory Model

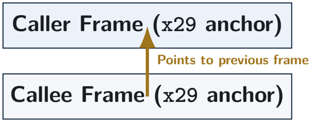
- The runtime stack is a dynamic memory region allocated in process address space.
- **Key Characteristics:**
  - Grows **downward** from high memory addresses toward low memory addresses.
  - Stack Pointer (sp) points to the current **lowest active address** (top of stack).
  - Operates on a Last-In, First-Out (LIFO) order.

Allocation vs. Deallocation

- **Push / Allocate:** Subtract from `sp` (`sub sp, sp, #32`).
- **Pop / Deallocate:** Add to `sp` (`add sp, sp, #32`).

Frame Pointer (x29) and Frame Chains

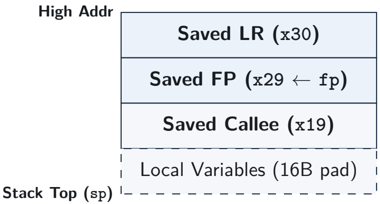
- The **Frame Pointer (`x29` / `fp`)** provides a fixed reference point within the activation frame for accessing arguments and local variables.
- **The Frame Pointer Chain:**
  - Each function saves the previous caller's `x29` on the stack.
  - `x29` is updated to point to this saved location.
  - Forms a linked list of stack frames back to `main()`.



Essential for runtime debuggers (e.g., GDB) to generate call stack backtraces.

Standard Activation Frame Structure

- An **Activation Frame** stores all data necessary for a single function execution instance.
- Saved in a standardized order:
  1. Saved Link Register (`x30`).
  2. Saved Frame Pointer (`x29`).
  3. Saved Callee Registers (`x19`–`x28`).
  4. Local Variables & Buffers.



Function Prologue Mechanics

- The **Prologue** is the setup code executed at the very beginning of a non-leaf function.
- Performs three standard actions:

Standard Prologue Template

```
my_func:
    // 1. Push FP and LR; allocate stack frame (32 bytes)
    stp   x29, x30, [sp, #-32]!

    // 2. Set Frame Pointer to bottom of frame link
    mov   x29, sp

    // 3. Save any callee-saved registers used in body
    str   x19, [sp, #16]
```

Pre-indexed `stp` combines stack allocation and saving `x29/x30` into a single instruction.

Function Epilogue Mechanics

- The **Epilogue** tears down the frame and restores state before returning.
- Reverses the exact sequence performed by the prologue:

Standard Epilogue Template

```
// 1. Restore saved callee registers
ldr   x19, [sp, #16]

// 2. Restore original FP and LR; release stack frame
ldp   x29, x30, [sp], #32

// 3. Return to caller
ret
```

Post-indexed `ldp` restores `x29`/`x30` and deallocates 32 stack bytes atomically.

Local Variable Allocation on the Stack

C Function with Local Array

```c
void process(void) {
    uint64_t buffer[2];
    buffer[0] = 10;
    buffer[1] = 20;
}
```

AArch64 Stack Allocation

```
process:
    // 16B (FP/LR) + 16B (buffer) = 32B
    stp  x29, x30, [sp, #-32]!
    mov  x29, sp

    mov  x0, #10
    str  x0, [sp, #16] // buffer[0]
    mov  x0, #20
    str  x0, [sp, #24] // buffer[1]

    ldp  x29, x30, [sp], #32
    ret
```

Local stack variables are referenced using positive offsets from sp or x29.

Stack Alignment and Padding Rules

- AAPCS64 mandates that `sp` **must be a multiple of 16 at all times**.
- If local variables require 20 bytes, allocation must be padded up to 32 bytes $(16 + 16)$.

Calculating Frame Size

$$\text{Total Frame Size} = \text{align16}(\text{Header (16B)} + \text{Callee Regs} + \text{Local Variables})$$

Requested Space	**Padded Frame Allocation**	**Instruction**
16 bytes	16 bytes	`sub sp, sp, #16`
24 bytes	32 bytes	`sub sp, sp, #32`
40 bytes	48 bytes	`sub sp, sp, #48`

PART 4

Worked Assembly Examples & Pat-terns

Worked Example 1: Minimal Leaf Function

C Code: `int64_t add_three(int64_t a, int64_t b, int64_t c)`

AArch64 Assembly Implementation

```
// Inputs: x0 = a, x1 = b, x2 = c
add_three:
        add x0, x0, x1    // x0 = a + b
        add x0, x0, x2    // x0 = (a + b) + c
        ret               // Return result in x0
```

- **Analysis:** Calls no subroutines; uses scratch registers `x0`–`x2`.
- No stack allocation required; executes in 3 instructions.

Worked Example 2: Non-Leaf Nested Function

C Code: `int64_t compute(int64_t x)`

```
int64_t compute(int64_t x) {
    return add_three(x, 10, 20);
}
```

AArch64 Assembly Implementation

```
// Input: x0 = x
compute:
        stp x29, x30, [sp, #-16]!    // Save FP and LR (x30)
        mov x29, sp                  // Set up frame pointer
        mov x1, #10                  // 2nd arg = 10
        mov x2, #20                  // 3rd arg = 20
        bl add_three                 // Call add_three (overwrites x30)
        ldp x29, x30, [sp], #16      // Restore FP and original LR
        ret                          // Return to caller
```

Worked Example 3: Recursive Factorial

C Code: `uint64_t factorial(uint64_t n)`

AArch64 Assembly Implementation

```
// Input: x0 = n
factorial:
        cmp x0, #1                   // Check n <= 1
        b.ls .Lbase_case             // Base case jump
        stp x29, x30, [sp, #-16]!    // Save FP and LR
        stp x19, xzr, [sp, #-16]!    // Save callee reg x19
        mov x19, x0                  // Preserve n in x19 across call
        sub x0, x0, #1               // Pass (n - 1) in x0
        bl factorial                 // Recursive call
        mul x0, x0, x19              // Result = factorial(n-1) * n
        ldp x19, xzr, [sp], #16      // Restore x19
        ldp x29, x30, [sp], #16      // Restore FP/LR
```

Worked Example 4: Passing > 8 Arguments

Passing 9th and 10th Parameters via Stack

Parameters 1–8 in x0–x7; parameters 9 and 10 pushed to stack.

Caller Side (Preparing Call)

```
// Push 10th and 9th args
mov  x8, #100
mov  x9, #200
stp  x9, x8, [sp, #-16]!

bl   func_with_10_args

add  sp, sp, #16 // Clean stack
```

Callee Side (Reading Stack Args)

```
func_with_10_args:
    // Args 1-8 in x0-x7
    // Arg 9 at [sp, #0]
    // Arg 10 at [sp, #8]
    ldr  x8, [sp, #0]  // 9th
    ldr  x9, [sp, #8]  // 10th
    ret
```

Common Function Linkage Bugs in Assembly

- **1. Overwriting Link Register:** Executing `bl` in a non-leaf function without saving `x30` destroys the return path, causing infinite loops or crashes.
- **2. Corrupting Callee Registers:** Modifying `x19`–`x28` without preserving them corrupts caller local variables.
- **3. Stack Misalignment:** Adjusting `sp` by 8 bytes instead of 16-byte multiples triggers hardware alignment faults.
- **4. Unbalanced Stack Pointer:** Modifying `sp` in body without restoring exact offsets leads to incorrect return targets.

Review and Self-Test Questions

Topic 6: Comprehensive Summary

- **Linkage Primitives:** `bl` saves return context $(\mathrm{PC}+4)$ into `x30` and jumps; `ret` loads `x30` back into PC.
- **AAPCS64 Register Contract:**
  - Parameters: Passed in `x0`–`x7`; returns in `x0`/`x1`.
  - Scratch Registers: `x0`–`x17` (Caller-saved).
  - Preserved Registers: `x19`–`x28` (Callee-saved).
- **Activation Frames:** Anchor frame base with `x29`; allocate stack space in 16-byte aligned blocks.
- **Prologue/Epilogue:** Standard pre/post-indexed `stp`/`ldp` instructions handle saving and restoring `x29`/`x30` cleanly.

Self-Test Questions: Linkage & AAPCS64

1. **Leaf vs. Non-Leaf Function Linkage:** Why can a leaf function return safely without ever writing to the stack, whereas a non-leaf function must save `x30` to memory?
2. **Register Preservation Contracts:** Suppose function `foo()` calls `bar()`. If `foo()` needs to maintain a loop counter value across the call, should it store that counter in `x9` or `x19`? Why?
3. **Parameter Passing Limits:** How are integer parameters handled when calling a C function declared with 10 scalar arguments: `void test(int a1, ..., int a10)`?

Self-Test Questions: Stack Frame Management

1. **Stack Allocation Alignment:** A non-leaf function saves `x29`/`x30` (16 bytes) and needs 20 bytes of local buffer space. What is the minimum frame size it must allocate on the stack to maintain AAPCS64 compliance?
2. **Prologue/Epilogue Verification:** Identify the bug in the following epilogue:
```
ldp x29, x30, [sp], #16
ldr x19, [sp, #16]
ret
```
3. **Frame Pointer Chains:** How does saving `x29` on the stack allow a runtime debugger like GDB to reconstruct the full sequence of function calls leading to a crash?